

Lab Questions – 21st Nov 2025

PART 1 — SQL Queries

Q1. Create a table named students with fields:

- stdid INT PRIMARY KEY
- stdname VARCHAR(50)
- age INT
- city VARCHAR(50)

Solution:

```
mysql> create table students(stdid INT PRIMARY KEY, stdname VARCHAR(50), age INT, city VARCHAR(50));
Query OK, 0 rows affected (0.06 sec)

mysql> desc students;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stdid | int           | NO   | PRI | NULL    |       |
| stdname | varchar(50)   | YES  |     | NULL    |       |
| age   | int           | YES  |     | NULL    |       |
| city  | varchar(50)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Q2. Insert the following records into the students table:

stdid	stdname	age	city
1	Rohan	20	Pune
2	Meera	22	Mumbai
3	Arjun	21	Delhi
4	Kavya	23	Pune
5	Neha	22	Kolkata

Solution:

```
mysql> INSERT INTO students (stdid, stdname, age, city) VALUES
-> (1, 'Rohan', 20, 'Pune'),
-> (2, 'Meera', 22, 'Mumbai'),
-> (3, 'Arjun', 21, 'Delhi'),
-> (4, 'Kavya', 23, 'Pune'),
-> (5, 'Neha', 22, 'Kolkata');
Query OK, 5 rows affected (0.08 sec)
Records: 5 Duplicates: 0 Warnings: 0
```

Q3. Display all student records.

Solution:

```
mysql> select * from students;
+-----+-----+-----+-----+
| stdid | stdname | age  | city  |
+-----+-----+-----+-----+
| 1     | Rohan   | 20   | Pune  |
| 2     | Meera   | 22   | Mumbai |
| 3     | Arjun   | 21   | Delhi |
| 4     | Kavya   | 23   | Pune  |
| 5     | Neha    | 22   | Kolkata |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Q4. Display only the name and age of all students.

Solution:

```
mysql> select stdname, age from students;
+-----+-----+
| stdname | age  |
+-----+-----+
| Rohan   | 20   |
| Meera    | 22   |
| Arjun    | 21   |
| Kavya    | 23   |
| Neha     | 22   |
+-----+-----+
5 rows in set (0.00 sec)
```

Q5. Display students who are from Pune.

Solution:

```
mysql> select * from students where city = 'Pune';
+-----+-----+-----+-----+
| stdid | stdname | age  | city  |
+-----+-----+-----+-----+
| 1     | Rohan   | 20   | Pune  |
| 4     | Kavya   | 23   | Pune  |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Q6. Display students whose age is greater than 21.

Solution:

```
mysql> select * from students where age > 21;
+-----+-----+-----+-----+
| stdid | stdname | age | city |
+-----+-----+-----+-----+
|      2 | Meera   | 22  | Mumbai |
|      4 | Kavya   | 23  | Pune |
|      5 | Neha    | 22  | Kolkata |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Q7. Display students in descending order of age.

Solution:

```
mysql> select * from students order by age desc;
+-----+-----+-----+-----+
| stdid | stdname | age | city |
+-----+-----+-----+-----+
|      4 | Kavya   | 23  | Pune |
|      2 | Meera   | 22  | Mumbai |
|      5 | Neha    | 22  | Kolkata |
|      3 | Arjun   | 21  | Delhi |
|      1 | Rohan   | 20  | Pune |
+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Q8. Count how many students belong to each city. (Use GROUP BY).

Solution:

```
mysql> select city, count(city) as total_students from students group by city;
+-----+-----+
| city | total_students |
+-----+-----+
| Pune | 2 |
| Mumbai | 1 |
| Delhi | 1 |
| Kolkata | 1 |
+-----+-----+
4 rows in set (0.00 sec)
```

Q9. Display students whose name starts with 'K'. (Use LIKE).

Solution:

```
mysql> select * from students where stdname like 'K%';
+-----+-----+-----+-----+
| stdid | stdname | age  | city |
+-----+-----+-----+-----+
|      4 | Kavya   |    23 | Pune |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Q10. Delete student whose stdid = 5.

Solution:

```
mysql> delete from students where stdid = 5;
Query OK, 1 row affected (0.03 sec)

mysql> select * from students;
+-----+-----+-----+-----+
| stdid | stdname | age  | city |
+-----+-----+-----+-----+
|      1 | Rohan   |    20 | Pune |
|      2 | Meera   |    22 | Mumbai |
|      3 | Arjun   |    21 | Delhi |
|      4 | Kavya   |    23 | Pune |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

PART 2 — ALTER COMMAND QUESTIONS

Q11. Add a new column contact VARCHAR(15) to the students table.

Solution:

```
mysql> ALTER table students Add contact varchar(15);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc students;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stdid | int           | NO   | PRI | NULL    |       |
| stdname | varchar(50)   | YES  |     | NULL    |       |
| age    | int           | YES  |     | NULL    |       |
| city   | varchar(50)   | YES  |     | NULL    |       |
| contact | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Q12. Modify the data type of city column to VARCHAR(100).

Solution:

```
mysql> ALTER table students MODIFY city varchar(100);
Query OK, 4 rows affected (0.14 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> desc students;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stdid | int           | NO   | PRI | NULL    |       |
| stdname | varchar(50)   | YES  |     | NULL    |       |
| age    | int           | YES  |     | NULL    |       |
| city   | varchar(100)  | YES  |     | NULL    |       |
| contact | varchar(15)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

Q13. Rename the column stdname to student_name.

Solution:

```
mysql> ALTER table students CHANGE stdname student_name VARCHAR(50);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Q14. Drop the column contact from the table.

Solution:

```
mysql> ALTER table students drop column contact;
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

Q15. Add a new column gender ENUM('M','F').

Solution:

```
mysql> ALTER TABLE students
-> ADD COLUMN gender ENUM('M','F');
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> desc students;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| stdid          | int           | NO   | PRI | NULL    |       |
| student_name   | varchar(50)   | YES  |     | NULL    |       |
| age            | int           | YES  |     | NULL    |       |
| city           | varchar(100)  | YES  |     | NULL    |       |
| gender         | enum('M','F') | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

PART 3 — JOIN PRACTICE

TABLES :

Table: students

stdid	student_name	city
1	Rohan	Pune
2	Meera	Mumbai
3	Arjun	Delhi
4	Kavya	Pune

Table: marks

stdid	subject	marks
1	Maths	88
2	Maths	76
3	Maths	92
5	Maths	67

INNER JOIN

Q16. Display student name and marks of only those students who have matching IDs in both tables. (Students without marks should not appear.)

Solution:

```
mysql> select student_name, marks from students s inner join marks m on s.stdid = m.stdid;
+-----+-----+
| student_name | marks |
+-----+-----+
| Rohan        | 88    |
| Meera        | 76    |
| Arjun        | 92    |
+-----+-----+
3 rows in set (0.00 sec)
```

LEFT JOIN

Q17. Display all students and their marks. (If marks not available, show NULL.)

Solution:

```
mysql> select s.stdid, student_name, city, subject, marks from students s left join marks m on s.stdid = m.stdid;
```

stdid	student_name	city	subject	marks
1	Rohan	Pune	Maths	88
2	Meera	Mumbai	Maths	76
3	Arjun	Delhi	Maths	92
4	Kavya	Pune	NULL	NULL

4 rows in set (0.02 sec)

RIGHT JOIN

Q18. Display all marks records along with student names. (If student doesn't exist in students table, show NULL.)

Solution:

```
mysql> select m.stdid, student_name, subject, marks from students s right join marks m on s.stdid = m.stdid;
```

stdid	student_name	subject	marks
1	Rohan	Maths	88
2	Meera	Maths	76
3	Arjun	Maths	92
5	NULL	Maths	67

4 rows in set (0.00 sec)

CROSS JOIN

Q19. Display all possible combinations of students and subjects. (Use CROSS JOIN between students and marks table to show every pair.) JOIN with Filtering

Solution:

```
mysql> select s.stdid, student_name, subject, marks from students s cross join marks m;
```

stdid	student_name	subject	marks
4	Kavya	Maths	88
3	Arjun	Maths	88
2	Meera	Maths	88
1	Rohan	Maths	88
4	Kavya	Maths	76
3	Arjun	Maths	76
2	Meera	Maths	76
1	Rohan	Maths	76
4	Kavya	Maths	92
3	Arjun	Maths	92
2	Meera	Maths	92
1	Rohan	Maths	92
4	Kavya	Maths	67
3	Arjun	Maths	67
2	Meera	Maths	67
1	Rohan	Maths	67

16 rows in set (0.00 sec)

Q20. Using INNER JOIN, display students who scored more than 80.

Solution:

```
mysql> select s.stdid, student_name, city, subject, marks from students s inner join marks m on s.stdid = m.stdid where m.marks > 80;
```

stdid	student_name	city	subject	marks
1	Rohan	Pune	Maths	88
3	Arjun	Delhi	Maths	92

2 rows in set (0.00 sec)